

Федеральное государственное образовательное бюджетное учреждение
высшего образования

**«Финансовый университет при правительстве
Российской Федерации»
(Финансовый университет)**

Колледж информатики и программирования

Группа: 4ИСИП-923

ДОКЛАД

На тему: Жизненный цикл HTTP-запроса и паттерны разработки

Принял

_____ А.С. Рыбаков

Исполнил

_____ П.И.Шаврина

Москва

2026

ОГЛАВЛЕНИЕ

Жизненный цикл HTTP-запроса.....	3
Первый этап.....	3
Второй этап.....	4
Третий этап.....	4
Четвертый этап.....	4
Пятый этап.....	5
Шестой этап.....	5
Седьмой этап.....	5
Восьмой этап.....	6
Девятый этап.....	6
Какими стандартами описываются версии http?.....	7
Разница версий http.....	8
Паттерны разработки.....	8
Паттерны в практике PHP-разработки.....	10
СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ.....	11

Жизненный цикл HTTP-запроса.

Жизненный цикл HTTP-запроса представляет собой последовательность этапов, которые происходят при отправке HTTP-запроса от браузера к серверу и получении ответа.

Этот процесс включает несколько последовательных этапов: разбор URL, поиск IP-адреса сервера через DNS, установку TCP-соединения, передачу HTTP-запроса и ответа, а также завершение соединения.

Каждый этап важен, потому что от него зависит, сможет ли браузер получить нужные данные и отобразить страницу.

Первый этап

Всё начинается с URL — универсального указателя ресурса. URL описывает, к какому ресурсу хочет обратиться браузер и каким способом это нужно сделать. Первая часть URL — это протокол, например HTTP или HTTPS. Протокол определяет правила, по которым будет происходить обмен данными между клиентом и сервером. HTTPS — это защищённая версия HTTP, которая шифрует передаваемую информацию. После протокола в URL обычно идут двоеточие и две косые черты, а затем — доменное имя. Доменное имя состоит из имени сайта и домена верхнего уровня. Домен верхнего уровня — это последняя часть имени, например .com, .org или .ru. Также в URL может присутствовать поддомен, который воспринимается как домен внутри домена. Например, поддомен может указывать на отдельный раздел сайта или на конкретный сервис.

Характеристика	URI	URL	URN
Назначение	Общий концепт идентификации	Локация и доступ	Именованье ресурса
Содержит метод доступа	Может содержать	Обязательно содержит	Не содержит
Пространство имен	Объединяет все	Определяется протоколом	Определяется схемой urn:

Второй этап

Далее в URL может быть указан порт. Порт — это номер, который помогает компьютеру понять, к какой именно службе на сервере нужно обратиться. Однако в большинстве случаев порт не указывается, потому что для HTTP и HTTPS существуют стандартные порты. Для HTTPS по умолчанию используется порт 443, а для HTTP — порт 80. Если порт не указан, браузер автоматически подставляет стандартное значение.

Третий этап

После порта в URL идёт путь к ресурсу. Путь указывает, какую именно страницу или файл запрашивает браузер. Если пользователь открывает главную страницу сайта, путь часто можно не указывать. Но если нужен конкретный раздел или документ, путь добавляется после косой черты. Кроме пути, URL может содержать параметры запроса. Параметры запроса — это пары «ключ-значение», которые передаются серверу для уточнения запроса. Например, с их помощью можно фильтровать содержимое, сортировать данные или указывать дополнительные условия. Также в URL может быть фрагмент. Фрагмент начинается с символа # и содержит идентификатор элемента на странице. Браузер использует фрагмент для автоматической прокрутки к нужному месту документа.

Четвертый этап

Когда URL разобран, браузеру необходимо найти сервер, на котором находится нужный ресурс. Для этого нужно определить IP-адрес сервера. IP-адрес — это уникальный идентификатор компьютера в сети. Чтобы узнать IP-адрес по доменному имени, используется Domain Name System, или DNS. DNS — это система доменных имён, которая сопоставляет доменные имена и IP-адреса. Когда браузеру нужно найти IP-адрес, он сначала проверяет локальный кэш. Операционная система хранит кэш IP-адресов, которые недавно были найдены через DNS. Благодаря этому повторные запросы выполняются быстрее и не требуют обращения к сети.

Пятый этап

Если IP-адрес не найден в локальном кэше, браузер отправляет DNS-запрос в сеть. Сначала этот запрос попадает на сервер резолвинга имён. Сервер резолвинга имён обычно принадлежит интернет-провайдеру. Он проверяет собственный кэш и, если адрес там есть, сразу возвращает его. Если сервер резолвинга имён не знает IP-адрес, он обращается к корневому серверу имён. Корневой сервер хранит информацию о доменах верхнего уровня. Он не знает точного IP-адреса нужного домена, но может подсказать, где искать дальше. Корневой сервер направляет запрос к серверу домена верхнего уровня. Сервер домена верхнего уровня знает, какие полномочные серверы имён отвечают за конкретные домены. Он перенаправляет запрос к полномочному серверу имён. Полномочный сервер имён — это сервер, который хранит точную информацию о домене, включая его IP-адрес. Он возвращает IP-адрес серверу резолвинга имён, а тот передаёт его компьютеру пользователя. После этого браузер знает, куда обращаться.

Шестой этап

Когда IP-адрес известен, браузер устанавливает соединение с сервером. Для этого используется TCP — Transmission Control Protocol, или протокол управления передачей. TCP — это сетевой протокол, который обеспечивает надёжную передачу данных между двумя компьютерами в интернете. TCP является основным механизмом доставки HTTP-запросов. Он передаёт информацию пакетами. Пакет — это небольшой фрагмент данных, который является частью большего сообщения. Пакеты объединяются вместе, образуя полное передаваемое сообщение.

Седьмой этап

Чтобы установить TCP-соединение, клиент должен инициировать подключение. Клиент отправляет серверу пакет синхронизации, который называется SYN. Сервер отвечает клиенту пакетом подтверждения синхронизации, который называется SYN-ACK. Затем клиент отправляет серверу подтверждение, что он получил ответ. Этот процесс называется

трёхэтапным рукопожатием, или three-way handshake. После успешного завершения трёхэтапного рукопожатия два компьютера считаются подключёнными друг к другу по TCP. Теперь они могут начать передавать HTTP-сообщения.

Восьмой этап

Браузер отправляет HTTP-запрос серверу. Запрос содержит метод, путь, версию протокола и дополнительные заголовки. Сервер обрабатывает запрос и формирует HTTP-ответ. Ответ содержит статус-код, заголовки и тело ответа. Тело ответа может содержать HTML-код страницы, стили, скрипты, изображения или другие данные. Браузер получает ответ и начинает обрабатывать его: строит DOM-дерево, загружает дополнительные ресурсы, применяет стили и выполняет скрипты. Так страница постепенно отображается пользователю.

Девятый этап

Когда пользователь уходит с веб-сайта или соединение больше не нужно, TCP-соединение завершается. Для завершения соединения каждая из сторон отправляет другой стороне пакет завершения, который называется FIN. Другая сторона подтверждает получение этого пакета. После этого соединение считается закрытым. Завершение соединения важно, потому что оно освобождает ресурсы на клиенте и на сервере.

Таким образом, жизненный цикл HTTP-запроса включает разбор URL, поиск IP-адреса через DNS, установку TCP-соединения с помощью трёхэтапного рукопожатия, передачу HTTP-запроса и ответа, а также завершение TCP-соединения. Каждый этап играет свою роль в обеспечении надёжной и упорядоченной передачи данных между браузером и сервером. Понимание этого процесса помогает лучше представлять, как работает интернет и какие механизмы обеспечивают загрузку веб-страниц.

Какими стандартами описываются версии http?

RFC (Request for Comments) — это серия пронумерованных документов, которые содержат технические спецификации, стандарты и обсуждения,

связанные с Интернетом и компьютерными сетями. Название можно перевести как «заявка (запрос) на отзывы», «тема для обсуждения» или «рабочее предложение».

Сегодня RFC — основной механизм формальной стандартизации в среде IETF (Internet Engineering Task Force, Инженерный совет Интернета) и смежных организаций. Эти документы лежат в основе работы ключевых протоколов, таких как TCP/IP, HTTP, SMTP, DNS и многих других.

HTTP/1.0 - В мае 1996 года для практической реализации HTTP был выпущен информационный документ RFC 1945, что послужило основой для реализации большинства компонентов HTTP/1.0.

HTTP/1.1 - Современная версия протокола; принята в июне 1999 года. Новым в этой версии был режим «постоянного соединения»: TCP-соединение может оставаться открытым после отправки ответа на запрос, что позволяет посылать несколько запросов за одно соединение. Клиент теперь обязан посылать информацию об имени хоста, к которому он обращается, что сделало возможной более простую организацию виртуального хостинга.

HTTP/2 является первой новой версией HTTP с версии HTTP 1.1, которая была стандартизирована RFC 2616 в 1999. Рабочая группа представила протокол HTTP/2 на рассмотрение IESG как Proposed Standard в декабре 2014 и IESG утвердила его к публикации как Proposed Standard 17 февраля 2015 года. Спецификация HTTP/2 была опубликована как RFC 7540 в мае 2015 года.

HTTP 3.0 - это интернет-проект, отличный от предыдущих версий HTTP, которые были / являются документами запроса комментариев (RFC) Инженерной рабочей группы по Интернету (IETF). Его первый проект был опубликован в 2020 году.

Основное различие между HTTP 2.0 и HTTP 3.0 заключается в используемом протоколе транспортного уровня. В HTTP 2.0 используются TCP-соединения с поддержкой TLS (HTTPS и HTTP) или без нее. HTTP 3.0, в свою очередь, основан на QUIC (Quick UDP Internet Connections).

Если вкратце, QUIC — это протокол транспортного уровня с собственным мультиплексированием и встроенным шифрованием. QUIC обеспечивает быстрое установление соединения, а также позволяет снизить задержку при соединениях с потерями и медленных соединениях.

Помимо потенциальных преимуществ, унаследованных от QUIC, еще одной важной особенностью HTTP 3.0 является то, что он всегда создает зашифрованные соединения. Таким образом, он аналогичен использованию HTTPS в современном HTTP 2.0.

Разница версий http

Характеристика	HTTP/0.9 (1991)	HTTP/1.0 (1996)	HTTP/1.1 (1997)	HTTP/2.0 (2015)	HTTP/3.0 (2022)
Транспорт	TCP	TCP	TCP	TCP	QUIC (на базе UDP)
Формат	Текстовый	Текстовый	Текстовый	Бинарный	Бинарный
Соединение	Короткое	Короткое	Длительное (keep-alive)	Длительное	Длительное
Мультиплексирование	Нет	Нет	Нет (только конвейеризация)	Да (несколько запросов в одном соединении)	Да (независимые потоки)
Блокировка начала очереди (HOL)	Есть	Есть	Есть	На уровне TCP (пакет блокирует все потоки)	Решена (пакет блокирует только свой поток)
Сжатие заголовков	Нет	Нет	Нет	HPACK	QPACK
Server Push	Нет	Нет	Нет	Да (устарел в браузерах)	Нет (удален из-за сложности)
Установка соединения	3-RTT	3-RTT	3-RTT	2-3 RTT	1-RTT (или 0-RTT для повторных)
Миграция соединения	Нет	Нет	Нет	Нет	Да (при смене сети Wi-Fi→LTE)
Безопасность	Нет	Нет	Опционально (TLS)	Опционально (TLS)	Обязательно (TLS 1.3)

Паттерны разработки.

Начните с базового определения: **паттерны проектирования** — это проверенные решения типовых задач, с которыми разработчики сталкиваются снова и снова. Это не готовый код, а скорее «чертёж» или «рецепт». Они помогают структурировать код, делать его гибким, поддерживаемым и тестируемым.

Классификация паттернов:

1. Порождающие паттерны

Эти решают задачи создания объектов.

Singleton («Одиночка»). Гарантирует, что у класса будет только один экземпляр, и предоставляет глобальную точку доступа к нему.

Пример: подключение к базе данных или логгер. Важное предупреждение: злоупотребление Singleton может привести к жёсткой связности и другим проблемам.

Factory Method («Фабричный метод»). Определяет интерфейс для создания объектов, но позволяет подклассам менять тип создаваемых объектов.

Abstract Factory («Абстрактная фабрика»). Создаёт семейства связанных объектов, не привязываясь к конкретным классам.

Builder («Строитель»). Позволяет создавать сложные объекты пошагово. Помогает избежать «телескопического конструктора».

Prototype («Прототип»). Позволяет создавать новые объекты путём клонирования (копирования) существующих, а не конструировать их с нуля.

2. Структурные паттерны

Они определяют, как классы и объекты собираются в более крупные структуры.

Adapter («Адаптер»). Позволяет объектам с несовместимыми интерфейсами работать вместе.

Decorator («Декоратор»). Динамически добавляет объектам новую функциональность, оборачивая их в «обёртки».

Facade («Фасад»). Предоставляет простой интерфейс к сложной системе классов, библиотеки или фреймворка. Пример из Laravel: статические прокси к сервисам контейнера.

Proxy («Заместитель», «Прокси»). Подставляет вместо реальных объектов специальные объекты-заменители. Эти объекты перехватывают вызовы к оригинальному объекту.

Flyweight («Легковес»). Позволяет вместить больше объектов в оперативную память, разделяя общее состояние между ними, а не храня одинаковые данные в каждом объекте.

3. Поведенческие паттерны

Они управляют распределением обязанностей и алгоритмами между объектами.

Observer («Наблюдатель»). Создаёт механизм подписки: одни объекты следят за событиями, происходящими в других. Широко используется в фреймворках (например, события моделей в Laravel).

Strategy («Стратегия»). Определяет семейство алгоритмов, инкапсулирует каждый из них и делает их взаимозаменяемыми во время выполнения программы. Пример: разные драйверы для кэширования, аутентификации или отправки писем.

Command («Команда»). Инкапсулирует действие и его параметры в отдельный объект. Это позволяет передавать запросы как аргументы, ставить их в очередь, логировать, поддерживать отмену операций.

Chain of Responsibility («Цепочка обязанностей»). Построает цепочку объектов для последовательной обработки запроса. Если один объект не может обработать запрос, он делегирует его следующему.

Iterator («Итератор»). Даёт способ последовательно обходить элементы составных объектов, не раскрывая их внутреннего представления.

~ Паттерны в практике PHP-разработки
MVC (Model-View-Controller) — архитектурный фундамент почти для каждого PHP-веб-фреймворка.

~ Dependency Injection (DI) и IoC-контейнеры — для внедрения зависимостей, что повышает тестируемость и развязку кода.

~ Repository («Репозиторий») — абстрагирует доступ к данным, отделяя бизнес-логику от базы данных и упрощая замену хранилища.

~ Active Record — модели, которые оборачивают строки из базы данных с методами query и save (например, Eloquent в Laravel).

СПИСОК ИСПОЛЬЗОВАННОЙ ЛИТЕРАТУРЫ

1. <https://habr.com/ru/companies/ruvds/articles/720704/?ysclid=mumhanxwkf407842999>
2. <https://ru.wikipedia.org/wiki/HTTP/3>
3. <https://ru.wikipedia.org/wiki/HTTP?ysclid=mumibsjd5913003868>
4. <https://ru.wikipedia.org/wiki/HTTP?ysclid=mumho6mdeb197740022>
[#Основные_механизмы_протокола](#)
5. <https://practicum.yandex.ru/blog/patterny-v-programmirovanii/?ysclid=mumqrhh03526934046>
6. <https://bfometr.ru/blog/rfc?ysclid=mums0mv13223589416>